

GEORISK-SPARK

Real-Time Geospatial Supply Chain Disruption and Route Intelligence

TEAM PROJECT GUIDE | 12-WEEK SEMESTER | FOUR OWNERS

This guide defines the project the team will build, how the four workstreams fit together, and how teammates can begin contributing. It is an implementation plan and onboarding document; the code repository currently contains scaffold only.

Goal: use H3 indexing and Spark streaming to turn synthetic vessel and hazard events into explainable risk and congestion outputs, then demonstrate delay estimates, safer-route comparisons, and a dashboard.

Problem

Logistics teams may learn about weather or port congestion after a vessel is already affected. Repeatedly testing millions of GPS points against complex polygons can be costly. The project uses H3 cells as a spatial index: assign a cell to each vessel point and cover each hazard polygon with cells, then compare cell IDs to find candidate exposure efficiently.

Semester outcomes

- A repeatable synthetic feed for vessel telemetry and time-bounded weather alerts.
- A Spark Structured Streaming pipeline that maps events to H3 cells, emits candidate risk alerts, and calculates unique vessel counts in 15-minute windows.
- An AWS S3 data lake with Parquet and Athena queries, a demonstration delay model, a risk-weighted route comparison, and an interactive dashboard.
- A team-built system whose data contracts, handoffs, limits, cloud operation, and results can be explained by all four members.

Scope and limits

Use synthetic events and a small graph. Predictions, alerts, and route comparisons demonstrate a data workflow; they are not validated forecasts or navigational guidance. H3 overlap is approximate and may need exact geometry refinement near hazard boundaries.

1. Target architecture

Build locally first, then run the same event contracts through AWS. The repository does not yet implement these pipeline components.

PRODUCE	INGEST	PROCESS	STORE	ANALYZE
Synthetic generator Telemetry + alerts	Kinesis Data Streams Two event streams	AWS Glue Spark streaming H3, alerts, 15-min density	Amazon S3 Date-partitioned Parquet	Athena + MLlib Routes + Streamlit

Data flow

Step	Planned behavior	Output
1. Generate	Create deterministic vessel positions and weather-alert polygons. Include UTC event time, stable IDs, severity, and alert expiry.	Versioned JSON events
2. Ingest	Publish telemetry and weather alerts to separate Kinesis streams. Keep a local replay adapter for development without AWS.	Kinesis records
3. Process	AWS Glue Spark Structured Streaming validates events, maps points and hazard coverage to H3, matches active hazards, deduplicates vessel/window/cell rows, and computes density.	Risk events + density rows
4. Store	Write raw and processed outputs as Parquet in S3. Register schemas and locations in the Glue Data Catalog.	Queryable history
5. Use	Query history with Athena; train and evaluate an MLlib delay baseline; compare route options; visualize status in Streamlit.	Analysis + demo

Storage rules

- Use event date as the principal S3 partition. Keep H3 IDs as columns rather than creating high-cardinality partitions.
- Keep raw events and derived outputs separately discoverable. Include schema version and H3 resolution in outputs.
- Use event-time windows and expire hazard alerts based on their valid-until time.

H3 is the candidate-matching index. A cell equality match is not proof that a point lies inside the exact polygon; document coverage behavior and refine boundary cases if needed.

2. Four-person work split

Each owner maintains the layer notes, code interfaces, and handoff. Review event and output contracts together before implementation branches grow independently.

Owner	Responsibility	First implementation deliverable	Handoff
Member 1 Ingestion	Synthetic event generation, repeatable replay, validation, and Kinesis publishing.	Deterministic generator and producers for telemetry and alert streams.	Deliver contract-valid events and publish/reject counters to Member 2.
Member 2 Spatial processing	Spark Structured Streaming, H3 point/polygon coverage, risk alerts, active hazard expiry, and density.	Glue streaming job with checkpointing and risk/density outputs.	Consume Member 1 contracts and deliver stable output schemas to Member 3.
Member 3 Storage + analytics	S3/Parquet lake, Glue Catalog, Athena queries, and MLlib delay baseline.	Date-partitioned datasets, query examples, model metrics vs mean baseline.	Consume Member 2 outputs and expose result fields/query outputs to Member 4.
Member 4 Routes + dashboard	Small port/lane graph, risk-threshold alternative route, Streamlit map and metrics.	Route comparison and dashboard using stable analytics outputs.	Agree event and query fields with Members 2 and 3; return missing fields early.

Shared integration habits

- Freeze initial contracts by Week 2. Any breaking change requires agreement and a schema version update.
- Use one short-lived branch per cohesive task. Review and merge into main; commit and push after each integrated milestone.
- Integrate at least weekly. Exchange fixture records and output examples before polishing each component.
- Each member demonstrates their layer at the final presentation; all four should be able to explain the end-to-end path and its limits.

A handoff includes input fields, output fields, a runnable example when implementation exists, setup instructions, known limits, and the next owner.

3. Shared event contracts

JSON Schemas in `schemas/v1/` are the starting source of truth. They define required fields; each owner may propose refinements, but downstream consumers must be informed before a breaking change.

Contract	Key fields	Rules
Vessel telemetry	schema_version, event_id, event_time, vessel_id, latitude, longitude, speed_knots, heading_deg	UTC ISO 8601 event time; latitude -90..90; longitude -180..180; speed non-negative.
Weather alert	schema_version, event_id, alert_id, event_time, valid_from, valid_until, hazard_type, severity, geometry	Severity 1..5; Polygon or MultiPolygon; GeoJSON coordinate order is longitude, latitude.
Risk event	event_time, vessel_id, h3_index, h3_resolution, risk_alert, matching_hazards	One result per accepted telemetry record; retain matching alert IDs and severity.
Density metric	window_start, window_end, h3_index, h3_resolution, unique_vessel_count	15-minute event-time window; count distinct vessels per cell after deduplication.

H3 decisions for the first build

- Start with resolution 7 and retain the resolution in every result; use resolution 8 only as a documented comparison.
- Convert vessel latitude/longitude to a point cell. Convert each hazard polygon to a coverage set using one explicitly chosen H3 overlap rule.
- Keep active-alert validity in the risk decision; expired alerts should no longer flag vessels.
- Treat cell matching as a fast spatial filter. Add exact geometry refinement only if boundary accuracy is part of the accepted semester scope.

Delay model and route scoring

Create synthetic historical features and a transparent delay target based on congestion, hazard severity, and controlled noise. Use a time-based holdout and compare Spark MLlib Gradient-Boosted Trees with a mean baseline. For routing, score a small port/sea-lane graph with travel cost plus a documented risk penalty; avoid or penalize edges above the agreed risk threshold. Label outputs as simulated.

4. Twelve-week milestone plan

Deliver a thin end-to-end path before adding volume or model complexity. Each three-week phase ends with an integrated review.

Weeks	Focus	Evidence
1-3 Scaffold	Confirm contracts, assign owners, initialize repo, add stubs, schemas, setup/config guidance, and this guide.	Every member can locate their files, explain their input/output, and review the shared contracts. No pipeline implementation is expected yet.
4-6 Ingestion + spatial	Build generator and Kinesis publishing, then Glue streaming with H3 risk and deduplicated window density.	Short cloud run consumes generated events and writes inspectable output; expiry and checkpoint behavior are documented.
7-9 Lake + analytics	Write S3 Parquet, register tables, query with Athena, and train/evaluate delay baseline.	Queries return expected counts; held-out model metrics are compared with the simple baseline.
10-12 Route + demo	Implement route comparison and dashboard, integrate, rehearse, and prepare course presentation.	Scripted demo shows generated event through risk, density, storage, estimate, detour, and map; each owner explains limits.

Weekly cadence

Checkpoint	Team action
Plan	Choose one owned outcome per person; surface dependencies and confirm schema stability.
Integrate	Exchange example inputs/outputs and resolve timestamp, coordinate order, and partition decisions.
Review	Run what exists end-to-end, record blockers with owners, update docs, commit and push the milestone.

If time gets short, protect the stream-to-risk-to-storage-to-dashboard path. Reduce model sophistication or graph size before dropping integration time.

5. AWS operating plan

The repository currently has placeholders only. No AWS resource has been provisioned. Use the student account only after checking service access, current prices, region support, and expected run time.

Service	Planned role	Cost and setup practice
Kinesis Data Streams	Separate vessel and hazard ingestion streams.	Use modest demo volume and bounded retention; record configuration and stream names.
AWS Glue	Managed Spark Structured Streaming from Kinesis to processed output.	Run only during a planned test/demo; streaming jobs accrue charges while active. Configure S3 checkpoints and stop jobs afterward.
Amazon S3	Raw archive, checkpoints, processed Parquet, and Athena query output.	Separate prefixes and event-date partitions. Use a dedicated project bucket and teardown notes.
Glue Data Catalog	Table schema and location metadata for S3 datasets.	Keep schemas and table definitions versioned with code and docs.
Athena	SQL analytics against cataloged S3 data.	Use bounded, columnar queries and inspect scanned bytes.
Streamlit	Interactive presentation of map and analytic layers.	Use a local AWS profile or SSO; never embed keys in the dashboard or repo.

Security and cost controls

- Use least-privilege execution roles scoped to project streams, bucket prefixes, catalog tables, and logs.
- Keep secrets outside Git. `.env.example` contains names and safe placeholders only.
- Track each paid resource with an owner, purpose, run window, and teardown action.
- Check free-tier eligibility and pricing before provisioning; do not assume every service or duration is free.
- Keep synthetic outputs clearly labeled as demonstrations, not real-world forecasts.

AWS references

AWS Glue streaming ETL: <https://docs.aws.amazon.com/glue/latest/dg/add-job-streaming.html>

Athena SQL and S3 data catalogs: <https://docs.aws.amazon.com/athena/latest/ug/using-athena-sql.html>

H3 documentation: <https://h3geo.org/docs/>

6. What is in the repository now

The first project baseline is a scaffold, not a runnable data pipeline. It gives four teammates places to work and a shared contract to build against.

Path	Purpose
<code>`src/georisk/ingestion/`</code>	Member 1 notes and TODO entry points for deterministic generation and Kinesis publishing.
<code>`src/georisk/processing/`</code>	Member 2 notes and TODO entry points for H3 helpers and the Glue streaming job.
<code>`src/georisk/storage_analytics/`</code>	Member 3 notes and TODO entry points for S3 lake output and the delay model.
<code>`src/georisk/route_dashboard/`</code>	Member 4 notes and TODO entry points for route comparison and Streamlit.
<code>`schemas/v1/`</code>	Versioned JSON Schemas for telemetry, weather alerts, risk events, and density metrics.
<code>`infra/aws/`, `config/`, `tests/`</code>	AWS setup guidance, safe configuration notes, and a validation plan. No infrastructure or test implementation is present.
<code>`output/pdf/`</code>	This team guide with architecture, responsibilities, schedule, contracts, and cloud guidance.

How to contribute

- Choose one workstream and read its README and the shared schemas before editing.
- Implement only that workstream's TODO functions first; agree interface changes with dependent owners.
- Add dependencies when an implementation requires them; do not install AWS services or embed credentials in the repo.
- Add validation with each feature and update the workstream README and this guide when behavior changes.
- Commit focused changes and push after each integrated milestone so teammates can review and build on the same state.

Scaffold acceptance

A new teammate can clone the repository, identify their assigned files, locate input/output contracts, understand their handoff, and see the first milestone. The placeholder modules are expected to raise ``NotImplementedError``; there is no local demo command yet.

Next build step: Member 1 implements deterministic event generation against the checked-in schemas; Member 2 reviews the fields and sample format before the Kinesis handoff is added.